

سوال ۱.

ابتدا توضیح دهید clustered یا unclustered بودن شاخص به چه معناست و اهمیت آن چیست؟

توضیح:

در ذخیره سازی، در صورتی که ترتیب دیتاریکورد ها با ترتیب data entry ها یکسان و یا تقریباً یکسان باشد (نزدیک باشد) clustered است و اگر چنین نباشد و ترتیب دیتا ریکورد ها با ترتیب دیتا انتری ها تا حد خوبی تفاوت داشته باشد، unclustered است.

اهمیت:

اگر بخواهیم دیتا ریکورد ها را از روی index آنها بازیابی کنیم، به لحاظ هزینه ی بازیابی بسیار تفاوت ایجاد می کند. و از لحاظ تعداد فرایند I/O بسیار تفاوت ایجاد میکنند. و با توجه به شرایط باید ببینیم که ذخیره کردن به صورت clustered بهتر است یا unclustered.

سپس در هر یک از سناریوهای زیر، توضیح دهید بهتر است از شاخص clustered استفاده شود یا unclustered یا فرقی ندارد؟

- یک شاخص درختی alt. 2 داریم و بیشترین پرسوجو از نوع search equality است .

Clustered بهتر است. برای درخت alt2 بهتر است در این شرایط از clustered استفاده کنیم.

در حالت clustered از $D * (\log 1.5 B)$ عملیات IO باید استفاده کنیم و در حالت unclustered به $1.5 B * \log 1.5 B + 1$ نیاز داریم.

- یک شاخص بر روی سن افراد می خواهیم تعریف کنیم و بیشترین پرسوجو از نوع search range بر روی سن افراد است. قبلاً یک شاخص clustered بر روی محل تولد تعریف کرده ایم.

در این شرایط نیز clustered بهتر است. با توجه به این نکته که در هر فایل فقط یک شاخص clustered می توانیم داشته باشیم. و ایندکس را به " محل تولد و سن" تغییر دهیم. تا سن های مشابه کنار هم باشند و search range بهتر می شود.

سوال ۲.

قسمت ۱) شاخص hash را توضیح دهید. بررسی کنید این شاخص برای search equality بهتر عمل میکند یا search range؟

در index های hash based از hash function استفاده می شود.

- با استفاده از hash function ، با توجه به search key مربوطه، باکت مربوط به یک ریکورد را پیدا می کنیم. این شاخص hash – based برای equality بسیار خوب عمل می کند. در این جا فقط به دو عملیات ورودی / خروجی نیاز داریم.
- هر ایندکس مجموعه ای از ریکورد هاست. هر باکت یک page primary دارد و نیز شامل یک یا چند صفحه ی اور فلو نیز می تواند باشد.

- برای range search: به B تا عملیات ورودی/ خروجی نیاز داریم.

قسمت ۲) تفاوت شاخص alt. 2 و alt. 3 را با رسم شکل توضیح دهید. مختصر توضیح دهید در هر پایگاه داده چند شاخص از نوع ۱. alt2, alt3, و alt3 میتوانیم داشته باشیم؟

در alt3 هر search key شامل لیستی از rid هایی می شود که شامل آن search key هستند. فضای کمتری میگیرد و از alt2 کمی فشرده تر است. و بهتر از فضا استفاده می کند. ولی در عوض طول دیتا انتری ها در اینجا متفاوت می تواند باشد. زیرا تعداد رکورد های مربوط به هر کلید لزوما برابر نیستند. و این تفاوت می تواند آنقدر زیاد باشد که تعادل تعدادی را از بین ببرد. این موضوع روی کارایی شاخص اثر منفی میگذارد.

مثلا در بین دانشجویان دانشگاه طول دیتا انتری سن = ۲۱ احتمالا بسیار بیشتر از سن = ۴۵ خواهد بود. و این موضوع روی کارایی شاخص اثر سوء دارد.

k1	[rid1, rid2, rid3, ...]
k2	[rid3, rid4, rid5, ...]

Alt2: هر کدام از data entry ها دارای rid و search key هستند. برای هر رکورد کلید جستجو در کنار اشاره گر record id به آن رکورد را نگه می دارد.

k1	rid1	k2	rid3
k1	rid2	k2	rid4

نقطه اشتراک این دو alt2 و alt3 این است که شامل data entry هایی هستند که به data record ها اشاره می کنند.

در نهایت حداکثر یک index از alternative1 می توانیم داشته باشیم. علت این است که در غیر این صورت، دیتای ما به صورت تکراری ذخیره می شود زیرا record id نیست و به جای آن data record را داریم.

اما در alternative2 و alternative3 تعداد محدودیتی ندارد.

قسمت ۳) آیا استفاده از شاخصها میتواند موجب کاهش سرعت عملیات بروزرسانی در پایگاه داده شود؟ با ذکر مثال توضیح دهید.

بله. استفاده از شاخص ها سبب کاهش سرعت عملیات بروزرسانی در پایگاه داده میشود. در صورتی که به درستی جستجو نشود یا، از search key باید استفاده شود و یا در صورتی که بخواهیم یک attribute که شاخص هست را اپدیت کنیم باید هزینه ی بالایی پرداخت کنیم.

در واقع اگر در یک جستجو نتوانیم از شاخص ها استفاده کنیم و همچنین یک خصیصه که شاخص است را اپدیت کنیم. در این وضعیت نه تنها نمیتوانیم از ایندکس ها استفاده کنیم، بلکه آنها را باید update کنیم.

UPDATE Company

SET

Salary = 6000

WHERE

Rating <3 AND name = "Pajooheshgah";

سوال ۳.

قسمت ۱) دو روش directory page و list linked را برای پیاده سازی file heap در نظر بگیرید. تفاوت این دو روش را به اختصار توضیح دهید.

:directory Page

در این روش، تعدادی دیرکتوری را به صورت لیست پیوندی قرار می دهیم. و در هر کدام از آنها، اطلاعات مربوط به صفحات یعنی پوینتر به صفحه و جا داشتن یا نداشتن را مینویسیم. در اینجا، میزان فضای خالی را نیز نگه میداریم و با این روش، مشکل record ها با طول متغیر حل میشود.

لیست پیوندی:

از دو لیست پیوندی دو طرفه استفاده میکنیم. یکی برای صفحات پر و دیگری برای صفحات خالی.

هرگاه داده ای از صفحات پر حذف شد و صفحه جا داشت آن را به لیست دیگر انتقال می دهیم. در حین نوشتن اگر صفحه پر شود، آن را به لیست پر ها اضافه میکنیم. به این ترتیب یافتن داده ها و صفحات خالی به سرعت و سهولت انجام می شود. فقط اگر طول رکورد ها ثابت نباشد. در این حالت ممکن است فضای کوچکی در انتهای صفحات باقی بماند و همه صفحات در لیست خالی ها جا بگیرند.

قسمت ۲) در پایگاه داده‌های ۱۸ صفحه داده داریم و ظرفیت بافر مورد استفاده نامحدود است. پایگاه داده بصورت زیر است و file heap استفاده میکنیم. برای پرسش داده شده، بررسی کنید در بدترین حالت به چند عملیات IO نیاز است؟

Student(sld INTEGER, age INTEGER, name VARCHAR(20), email VARCHAR(20))

- Insert into Students(810100789, "name", "name@ut.ac.ir")

الف) پیاده سازی با directory page و هر دایرکتوری ظرفیت ۱۰ اشارهگر را دارد.

۱۸ IO -> در بدترین حالت، آدرس page عی که به اندازه کافی جای خالی دارد، در آخرین header هست.

۱ IO -> انتقال به دیسک

۲ IO -> برای update کردن header

۱۸ + ۲ + ۱ = ۲۱ IO نیاز است.

ب) استفاده از list linked و دوازده صفحه پر به همراه شش صفحه دارای فضای خالی.

۱ IO -> آوردن صفحه header به حافظه

۶ IO -> در بدترین حالت ۶ امین صفحه است

۳ IO -> اپدیت اشاره گر page، header و صفحه record جدید اضافه شده توی دیسک

۱ IO -> پس از نوشتن، پر میشود و باید به صف پر ها انتقال یابد

۱ IO -> پوینتر صفحه پیشین را نال میکنیم روی دیسک مینویسیم

۱۲ = ۱+۶+۳+۱ IO نیاز داریم.

سوال ۴.

plan-only-index چیست؟ مزیت این روش را بیان کنید.

در حالت index-only اطلاعات موجود در دیتا انتری ها برای جواب دادن به query کافیست. در این حالت، به واکنشی دیتا از حافظه نیازی نداریم. مزیت: مزین این روش این است که سبب کاهش هزینه پرسجو ها می گردد.

با در نظر گرفتن رابطه زیر، برای هر یک از پرسجو های داده شده، بهترین کلید شاخص برای پاسخ only-index را مشخص کنید.

product(pid, name, price, category, rate)

a) select P.category, MIN(P.price)
from product P
where P.rate > 5

group by P.category

در این حالت، index باید < category, rate, price > باشد. دیتا انتری ها در این روش در ابتدا بر اساس کتگوری مرتب شده اند. در صورتی که کتگوری یکسان داشته باشند، حال به صورت rate مرتب می شوند و انهایی که rate 5 دارند حذف میشوند. پرایس نیز در آخر جزو شاخص است چون مینیمم پرایس خواسته شده.

```
b) select AVG(P.price)
from product P
group by P.category
```

شاخص در اینجا < category , price > است. در ابتدا بر اساس کتگوری مرتب میشوند و کتگوری های یکسان کنار هم قرار می گیرند و میانگین پرایس محاسبه میشود.

سوال ۵.

روشهایی برای ذخیره رکوردها با طول متغیر بیان کرده و مختصراً توضیح دهید(بیان دو روش کافی است). یکی از این روش ها استفاده از offset است، مزایا و معایب این روش در برابر سایر روشها را بیان کنید.

در اینجا به توضیح دو روش آفست و header و delimiter می پردازیم:

در روش آفست، می توانیم همه ی field هایی که طولشان متغیر است را به آخر record انتقال دهیم. این امر سبب می شود برای دسترسی به فیلدی با طول ثابت، نیازی به اسکن همه ی ریکورد ها نباشد.

در صورتی که از delimiter ها استفاده نماییم، در واقع از خود جدا کننده ها در متن field ها نمیتوانیم استفاده کنیم. ولی در آفست چنین مشکلی وجود ندارد!

لازم به ذکر اسات همچنان برای دسترسی پیدا کردن به فیلد ها با طول متغیر، باید همه ی record ها را اسکن کنیم. البته انجام چنین کاری و استفاده از این روش، در مقایسه با روش header هزینه ی زمانی بیشتری دارد.

در واقع در روش header ، به آخر هر فیلدی متغیر اشاره دارد. در این روش نیازی به اسکن نداریم. ولی یک هدر اضافه داریم و همیشه نال بودن را با نال کردن پوینتر میتوانیم نشان دهیم.

- در روش delimiter باید کاراکتر خاص جدا کنیم. برای دسترسی به فیلد ها نیز به اسکن کلی نیاز داریم. و در صورتی که در خود متن فیلد ها از delimiter استفاده شود، کارمان کمی دشوار میشود.

سوال ۶.

پنج صفحه فیم در اختیار داریم. میخواهیم به ترتیب زیر و از چپ به راست به صفحات دستیابی داشته باشیم. استراتژیهای LRU و MRU را در نظر بگیرید و برای هر یک، ترتیب اختصاص فریمها به صفحات را مشخص کرده و در انتها rate hit را محاسبه کنید. تصور شود صفحات پس از دسترسی unpin شوند.

1-3-2-5-6-4-5-3-2-1-6-1-2-1-3-2-4-1-5

page	MRU	Hit	LRU	Hit
1	1 XXXX	-	1XXXX	-

Clustered hash - equality	$D * 2$
Clustered hash - range	$B * D \rightarrow$ اینجا کمکی نمی کند و باید همه ی رکورد های داده ای را بررسی کنیم
unclustered hash - equality	$(\log B/4 + 1) * D$
unclustered hash - range	$\log B/4 + \text{match records} + \text{match index page}$ به دلیل سورت نبودن دیتا باید همه ی آن هارا بررسی کنیم
Clustered tree - equality	$(1/10 * 100/40 * B = B/4 + 1) * D$
Clustered tree - range	$(\log 4/10 B + \text{match records} + \text{match page}) * D$

سوال ۸.

Level 0	ریداندنسی نداریم
Level 1	No data striping parallel read or write We have copy of it! -> if it destroys we have another one
Level 0+1	-parallel read, write - it has partitions -we have copy again!
Level 2	کد تصحیح خطا داریم pros of raid level 1+0 over level 1
Level 3	- performs so good in sequential data operations - has the parity option. -it offers impressive read, write speeds, esp. for large and heavy data applications از raid بالایی بهتر است. فقط یک بیت پرییتی داریم. عمل read و write شامل همه ی دیسک هاست.
Level 4	Striping یک بلاک است. یک دیسک پرییتی داریم. عملیات read را احتراز است و همه ی دیسک هارا درگیر نمیکنیم. برای عمل write کردن نیز فقط دو دیسک درگیر میکنیم.
Level 5	برای اینکه دیسک باتل نک نشود، بلاک های پرییتی را بین دیسک های موجود پخش میکنیم. درخواست write به صورت parallel انجام میشود.

5th RAID :
است

در اینجا کد داریم که جای خالی ها بقیه است

①

H
L
F
$p(M, -, T)$
H
O
E
$p(S, I, D)$
E

②

E
O
$p(F, R, D)$
M
E
T
$p(E, R, -)$
S
-

③

L
$p(L, D, -)$
R
-
-
$p(O, T, H)$
R
I
-

④

$p(H, S, L)$
-
D
T
$p(H, E, -)$
H
-
D
$p(b)$